

mDot: Compiling Dot-AT-Firmware using Mbed Studio

Overview

This document describes how to use the offline Mbed Studio Software to import, compile, and use the Dot-AT-Firmware source code which is the firmware installed at the factory. This example is using mDot module. The steps for the xDot legacy would be very similar, but the xDot legacy would use the libxDot library and "MultiTech xDot (STM32L151)" Target Hardware.

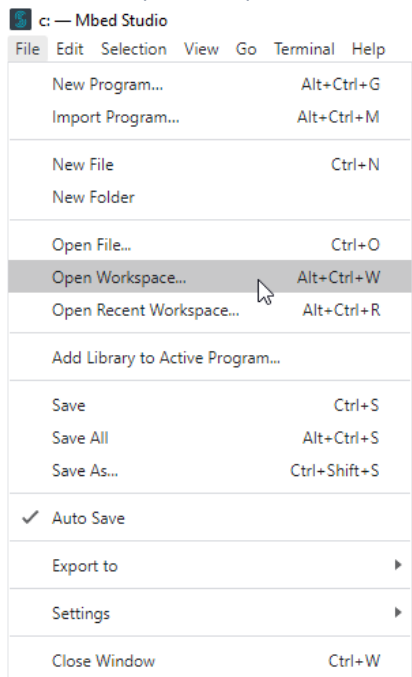
Requirements

- Have the Mbed Studio Software installed on you PC. This is a from IDE software available on the [mbed.com](https://os.mbed.com/studio/) website:
<https://os.mbed.com/studio/>
- Access to the internet on the PC running Mbed Studio, so that it can import the Dot-Examples project and the needed libraries.
- Mercurial install on PC.
<https://www.mercurial-scm.org/>
Make sure that "Add installation path to search path" is enabled during the installation.
- A mDot module.

Project Setup

Selecting a Project Folder

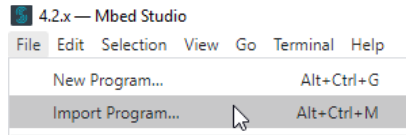
1. Open the Mbed Studio Software.
2. Click File > "Open Workspace..."



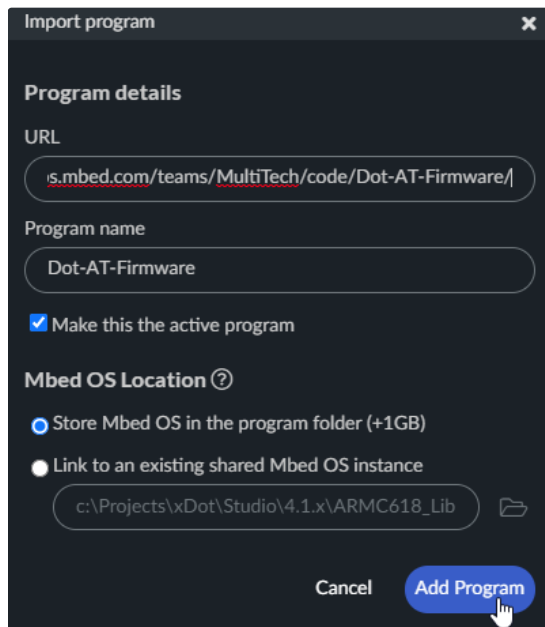
3. Browse to the project folder you want to import the Dot-AT-Firmware into, select the project folder and click "Select Folder".

Importing Dot-Examples Project

1. In Mbed Studio, click File > Import Program



2. In the “Import Program” dialog that pops up, enter “<https://os.mbed.com/teams/MultiTech/code/Dot-AT-Firmware/>” in the URL. The “Program name” should autofill, and you can change the name of the project if you desire. Leave the rest of the options at default and click the “Add Program” button.



3. After importing the Dot-AT-Firmware project, the “Active Program” should show as the Program Name you selected for Dot-AT-Firmware.

Change Project to a Commit That Supports the 4.1.x Library

The newer commits of the Dot-AT-Firmware were written for Dot library versions 4.2.x, which isn't fully supported on the mDot and xDot legacy modules. So we need to revert back to commit 35:e17e00b8e022 which supports the Dot 4.1.x libraries.

1. Close Mbed Studio.
2. Open a Command Prompt in windows and go to the Dot-AT-Firmware project folder.
3. While in the Dot-AT-Firmware project folder, enter the following command “hg update -C 35”. The -C will force the update and any changes will be lost, but at this point we haven't made any changes.
4. You can see what version the Dot-AT-Firmware project is currently at using the “hg summary”, verify that it shows “parent: 35:e17e00b8e022” as the version.

```
Command Prompt

F:\Projects\mDot\4.1.x\Dot-AT-Firmware>hg update -C 35
0 files updated, 0 files merged, 0 files removed, 0 files unresolved

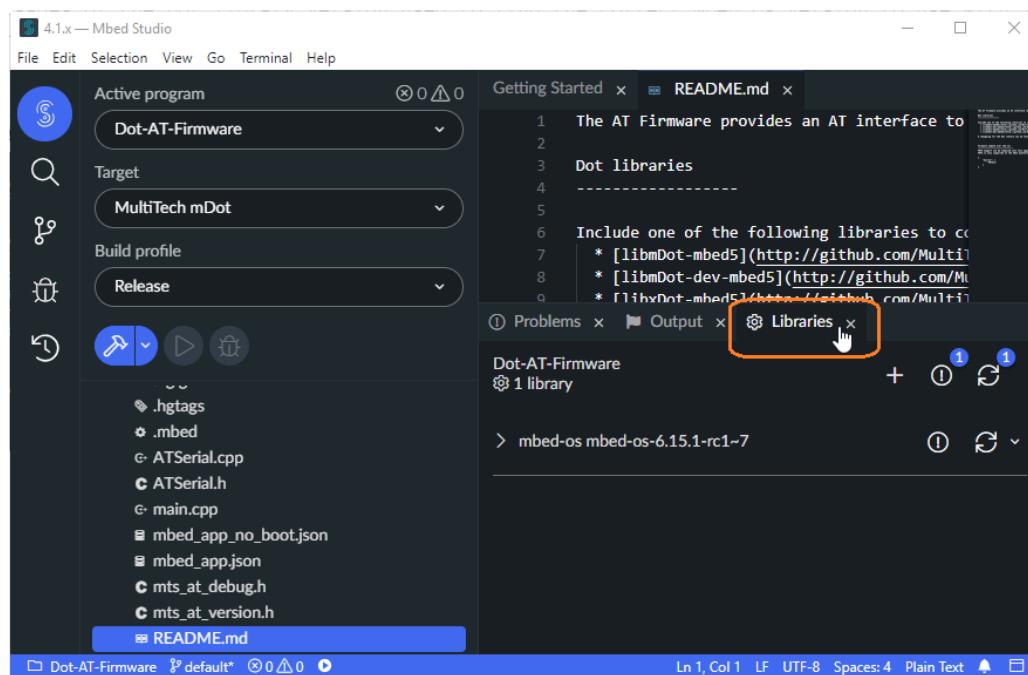
F:\Projects\mDot\4.1.x\Dot-AT-Firmware>hg summary
parent: 35:e17e00b8e022
Update AT Version to 4.1.5
branch: default
commit: 1 unknown (clean)
update: 7 new changesets (update)

F:\Projects\mDot\4.1.x\Dot-AT-Firmware>
```

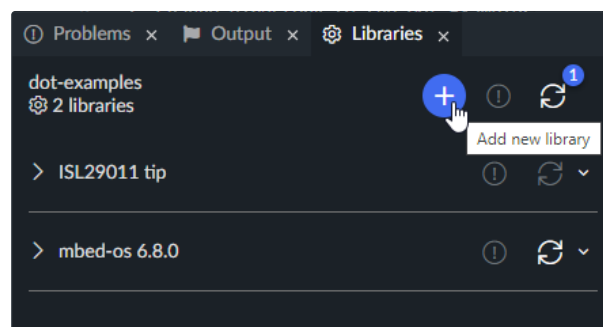
5. Close the Command Prompt windows and reopen Mbed Studio.

Adding mDot Library and Configuring Project

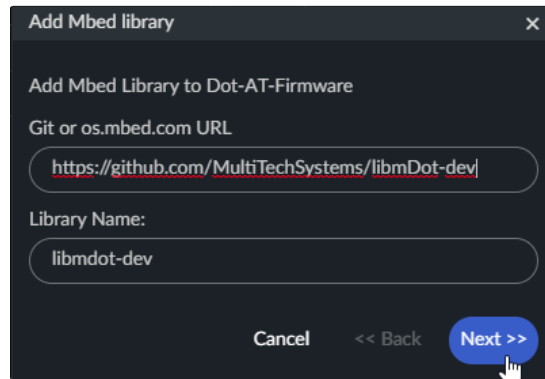
1. Select the “Libraries” Tab



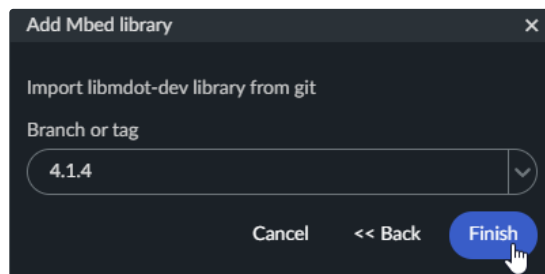
2. Click the “Add New Library” option



3. In the URL field enter the github URL link to the mDot library and click Next. Note that which library will depend on the version of the mDot library you wish to use. The 4.1.4 and earlier libraries were compiled with ARMCC 6.11, which isn't compatible the the compiler in Mbed Studio, so for 4.1.4 use the libmDot-dev library (<https://github.com/MultiTechSystems/libmDot-dev>) with the 4.1.4 tag version, this is the released 4.1.4 library compiled with a newer ARMCC version. For firmware version later than 4.1.4 use libmDot library (<https://github.com/MultiTechSystems/libmDot>). The screenshots will show using the 4.1.4 libmDot-dev library.



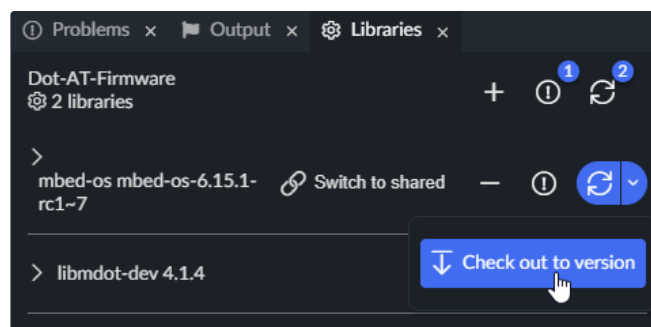
4. In the next dialog, select desired version. In this example we'll select 4.1.4 and then click Finish.



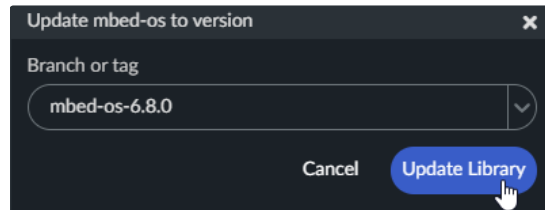
Note: Currently we don't recommend using the 4.2.x versions on the mDot or xDot Legacy models.

5. While still in the Mbed Libraries tab, select the version of mbed-os that matches the mDot library version was compiled with. Check the Github page for the libmDot library for what mbed-os version to use. Go to to releases, select the release, and then select (or hover over) the commit for that release and it will show the mbed-os version that release was compiled for.

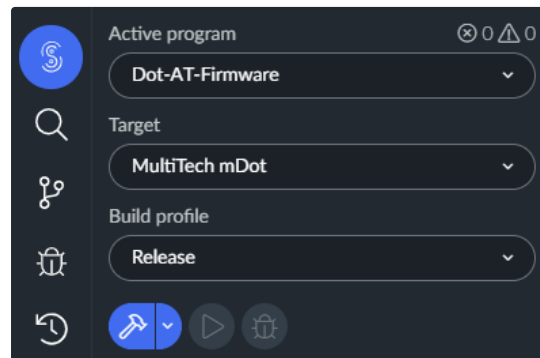
In this example we used 4.1.4 libmDot-dev library which should be compiled with mbed-os 6.8.0. If the mbed-os version in your project isn't at this version, then click the option to select the library version for the mbed-os library and select "Check out to version".



Scroll down and select the correct version, in this example it is "mbed-os-6.8.0", and then click "Update Library".



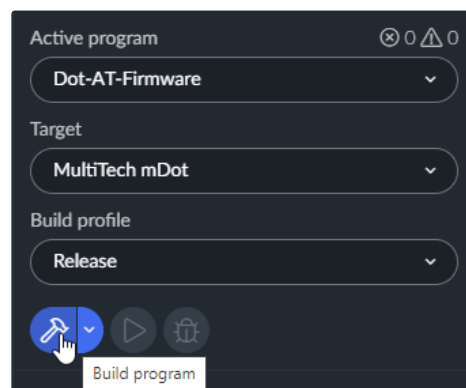
6. Set the "Target Hardware" to "MultiTech mDot", and set the "Build Profile" to "Release".



Compile Project

Verify that the project is setup properly and can successfully compile before making changes to the code.

To test the Project Settings, click on the "Build Project" icon (a hammer).



After several minutes the build should end with messages about the Image bin file location. Your compiled bin file will be at that location, usually in this will be "Dot-AT-Firmware\BUILD\MTS_MDOT_F411RE\ARM6\Dot-AT-Firmware.bin".

mbed-os\connectivity	350(+350)	0(+0)	0(+0)
mbed-os\drivers	3856(+3856)	0(+0)	1500(+1500)
mbed-os\events	1422(+1422)	0(+0)	0(+0)
mbed-os\features	18(+18)	0(+0)	0(+0)
mbed-os\hal	2604(+2604)	8(+8)	234(+234)
mbed-os\platform	4353(+4353)	80(+80)	416(+416)
mbed-os\rtos	1012(+1012)	0(+0)	8(+8)
mbed-os\targets	14454(+14454)	12(+12)	1464(+1464)
Subtotals	233277(+233277)	443(+443)	17357(+17357)
Total Static RAM memory (data + bss): 17800(+17800) bytes			
Total Flash memory (text + data): 233720(+233720) bytes			
Update Image: BUILD/MTS_MDOT_F411RE/ARM64\Dot-AT-Firmware_update.bin			
Image: BUILD/MTS_MDOT_F411RE/ARM64\Dot-AT-Firmware.bin			

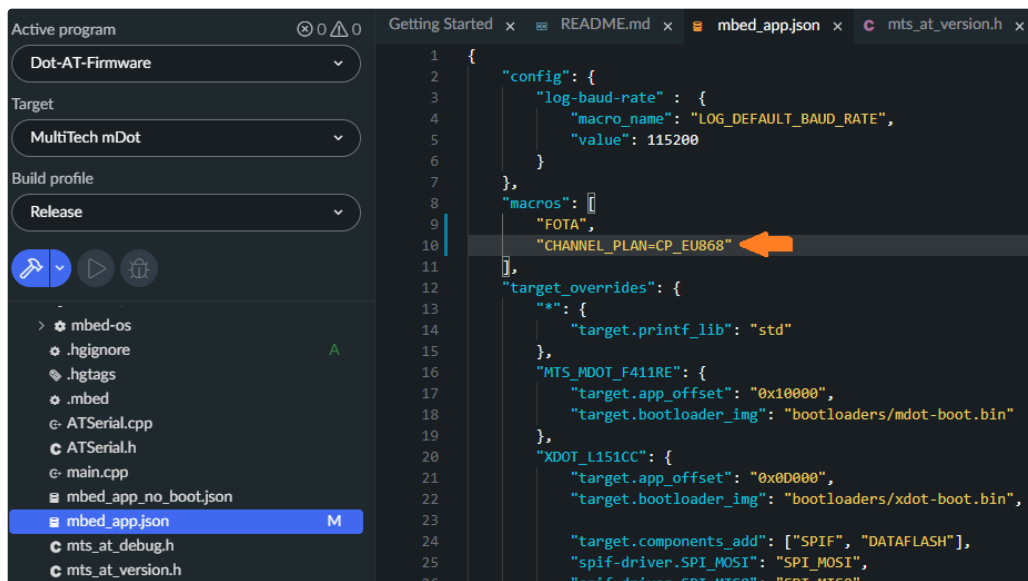
Setting the Channel Plan

By default the Dot-AT-Firmware will use the US915 channel plan, but this can be change by setting a CHANNEL_PLAN value in the mbed_app.json file macros section.

The project's mbed_app.json file is in the main folder of the project. Selecting this file will allow you to edit it.

Selecting a Non-Global Plan

To set the Dot-AT-Firmware to use a non-global plan, define the CHANNEL_PLAN as a specific Channel Plan in the mbed_app.json file's "macros" . For example, to use the EU868 non-global plan, define "CHANNEL_PLAN=CP_EU868" in "macros" in the mbed_app.json file:



Selecting a Global Channel Plan

You can choose a Global Plan by defining the CHANNEL_PLAN to CP_GLOBAL . The configured global plan can then be set by having a GLOBAL_PLAN definition in "macros" . For example, to use EU868 in a Global Plan, you would define "CHANNEL_PLAN=CP_GLOBAL" and "GLOBAL_PLAN=CP_EU868" :

```

1 {
2   "macros": [
3     "FOTA=1",
4     "CHANNEL_PLAN=CP_GLOBAL",
5     "GLOBAL_PLAN=CP_EU868"
6   ],
7   "target_overrides": {

```

Options for the `CHANNEL_PLAN` are :

CP_US915

CP_AU915

CP_EU868

CP_KR920

CP_AS923

CP_AS923_2

CP_AS923_3

CP_AS923_4

CP_AS923_JAPAN

CP_AS923_JAPAN1

CP_AS923_JAPAN2

CP_IN865

CP_RU864